LINUX DYNAMIC MEMORY ALLOCATION AND MEMORY MONITORING SYSTEM

1. Problem Statement

Efficient memory management is an important function of an operating system. During program execution, memory is continuously allocated and released according to the requirements of processes and applications. Frequent allocation and deallocation can result in inefficient memory utilization, memory fragmentation, and memory leaks.

However, the internal behavior of memory allocation is not always easy to observe directly. It is therefore useful to develop a system that can simulate and visualize memory allocation, deallocation, reallocation, and fragmentation.

The proposed project develops a **Linux-based Dynamic Memory Allocation and Memory Monitoring System** using C. The system maintains a logical memory pool divided into blocks and allows memory requests to be handled using different allocation strategies.

The system supports:

- Dynamic memory allocation
- Memory deallocation
- Memory reallocation
- First Fit allocation
- Best Fit allocation
- Worst Fit allocation
- Contiguous allocation
- Non-contiguous allocation
- Allocation ID tracking
- Memory status monitoring
- Internal fragmentation analysis
- External fragmentation analysis
- Memory utilization calculation

The project provides a practical way to understand how memory-management concepts studied in Operating Systems can be represented and analyzed through a working Linux application.

---

2. Objectives

The main objectives of the project are:

1. To implement dynamic memory allocation and deallocation in a Linux environment.
2. To maintain a record of allocated and free memory blocks during program execution.
3. To implement and demonstrate different memory allocation strategies:
    - First Fit
    - Best Fit
    - Worst Fit
4. To support both contiguous and non-contiguous memory allocation.
5. To implement memory reallocation for increasing and decreasing the size of an existing allocation.
6. To calculate and display memory utilization.
7. To identify and analyze external fragmentation caused by separated free memory blocks.
8. To demonstrate internal fragmentation using an allocation-unit approach in which requested memory may be rounded to the next allocation unit.
9. To provide allocation IDs so that individual memory blocks can be tracked and released.
10. To provide a simple menu-driven interface for interacting with the memory-management system.
11. To provide practical understanding of memory-management concepts and Linux system programming.

---

3. Selected Commands and Their Workflow

The project was developed and executed in the Linux/Ubuntu environment.

3.1 Opening the Project Directory

The project directory is accessed using:

cd ~/linux-memory-project

This changes the current working directory to the project folder.

3.2 Viewing Project Files

The `ls` command is used to view the files present in the project:

ls

The project contains files such as:

allocator.c

allocator.h

main.c

Additional backup/source files may also be present during development.

3.3 Compiling the Project

The C source files are compiled using GCC:

gcc -Wall -Wextra -std=c11 -o memory_system main.c allocator.c

Here:

- `gcc` is the GNU C Compiler.
- `-Wall` enables common compiler warnings.
- `-Wextra` enables additional warnings.
- `-std=c11` specifies the C11 standard.
- `-o memory_system` creates an executable named `memory_system`.
- `main.c` contains the main program and menu logic.
- `allocator.c` contains the memory-management implementation.

3.4 Running the Program

After successful compilation, the program is executed using:

./memory_system

The program then asks the user to enter the total memory size and select an allocation strategy.

3.5 Editing the Source Code

The Nano editor was used during development:

nano allocator.c

It was used to modify and test the allocator implementation.

3.6 Overall Workflow

The workflow of the project is:

Open Linux Terminal

↓

Navigate to Project Directory

↓

Compile Source Files using GCC

↓

Run memory_system

↓

Enter Total Memory

↓

Select Allocation Strategy

↓

Allocate / Free / Reallocate Memory

↓

Display Memory Status

↓

Analyze Fragmentation

↓

Observe Results

---

4. Implementation

The project is implemented in the C programming language and executed in a Linux/Ubuntu environment.

The system is divided into two main source components:

`main.c`

Responsible for:

- Starting the application
- Reading user input
- Displaying the menu
- Selecting allocation strategies
- Calling allocation/deallocation functions
- Displaying results

`allocator.c`

Responsible for the core memory-management logic, including:

- Memory-pool initialization
- Memory block management
- Allocation
- Deallocation
- Reallocation
- Allocation strategies
- Contiguous allocation
- Non-contiguous allocation
- Fragmentation analysis
- Memory-status reporting
- Cleanup

`allocator.h`

Contains declarations and definitions required by the allocator module.

4.1 Memory Pool

At program startup, the user specifies the total logical memory size.

For example:

Enter total memory size (KB): 100

The system creates a memory pool of 100 KB.

Initially, the entire memory pool is represented as one free block:



As allocation requests are received, this free memory is divided into used and free blocks.

4.2 Allocation Strategies

The system supports three allocation strategies.

First Fit

First Fit searches the available free blocks from the beginning and selects the first block that can satisfy the request.

Example:

Free blocks: 20 KB, 50 KB, 80 KB

Request: 40 KB

First Fit → 50 KB block

Best Fit

Best Fit searches the available free blocks and selects the smallest block that is large enough to satisfy the request.

Example:

Free blocks: 20 KB, 50 KB, 80 KB

Request: 40 KB

Best Fit → 50 KB block

The objective is to leave the smallest possible remaining free space among the suitable blocks.

Worst Fit

Worst Fit searches the available free blocks and selects the largest block that can satisfy the request.

Example:

Free blocks: 20 KB, 50 KB, 80 KB

Request: 40 KB

Worst Fit → 80 KB block

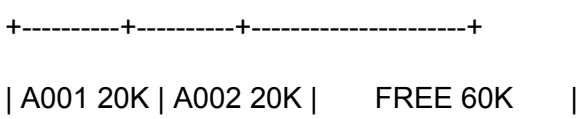
The remaining portion of the large block becomes available for future allocations.

4.3 Contiguous Allocation

In contiguous allocation, the requested memory is placed in one continuous block.

For example:

100 KB Memory



The allocation receives an Allocation ID such as:

A001

A002

A003

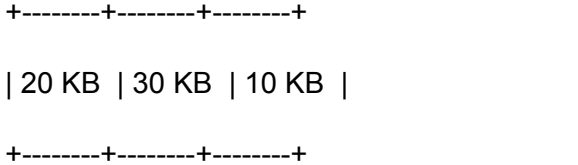
This allows individual allocations to be tracked and released.

4.4 Non-Contiguous Allocation

If a sufficiently large continuous free block is unavailable, the system can use multiple free segments to satisfy a memory request.

For example:

Free memory:



Request = 50 KB

The system can combine suitable free segments.

The output identifies the allocation as:

Allocation Type: NON-CONTIGUOUS

Segments Used: 2

This demonstrates the concept of allocating one logical request across multiple physical/logical memory segments.

---

5. Detailed Description of the Implementation

5.1 Memory Block Representation

Each memory block maintains information such as:

- Offset
- Size
- Requested size
- Free/used status
- Allocation ID
- Allocation type
- Link to the next block

Conceptually:

Memory Block

├── Offset

├── Size

├── Free / Used

├── Allocation ID

├── Allocation Type

└── Next Block

This allows the program to maintain a linked representation of the memory pool.

5.2 Allocation Process

When the user selects:

1. Allocate Memory

the system requests the required memory size.

For example:

Enter memory size (KB): 51

The system determines the appropriate allocation size according to the allocation-unit mechanism.

The output can be:

Requested Size : 51 KB

Allocated Size : 60 KB

Internal Waste : 9 KB

The system then records the allocation.

5.3 Allocation IDs

Every successful allocation receives a unique ID.

Example:

Allocation ID : A001

Allocation ID : A002

Allocation ID : A003

The ID can later be used to free or reallocate the corresponding memory.

For example:

Enter Allocation ID number:

ID: 2

This refers to:

A002

5.4 Deallocation

The user can select:

2. Free Allocation

and enter the allocation ID.

For example:

ID: 2

Allocation A002 freed successfully.

The corresponding memory becomes available again.

This allows the system to demonstrate how repeated allocation and deallocation can create separated free blocks.

5.5 Memory Status

The system provides a detailed memory-status display.

Example:

Block 1 | Offset: 0 KB | Size: 20 KB | USED | ID: A001

Block 2 | Offset: 20 KB | Size: 20 KB | FREE

Block 3 | Offset: 40 KB | Size: 20 KB | USED | ID: A003

Block 4 | Offset: 60 KB | Size: 40 KB | USED | ID: A004

The system also calculates:

Used Memory

Free Memory

Total Memory

Utilization

For example:

Used Memory : 80 KB

Free Memory : 20 KB

Total Memory: 100 KB

Utilization : 80.00%

5.6 Reallocation

The system supports resizing an existing allocation.

The user selects:

5. Reallocate Memory

and enters the allocation ID and new size.

Three important cases are demonstrated.

Shrinking

Example:

Old Allocated Size: 40 KB

New Requested Size: 20 KB

The allocation becomes smaller and the released space can be returned to the free-memory pool.

Output:

Operation: SHRINK

Expansion in Place

If sufficient adjacent free space exists, the allocation can be expanded without moving it.

Example:

Old Allocated Size: 20 KB

New Requested Size: 40 KB

Output:

Operation: EXPAND IN PLACE

Relocation

If the allocation cannot be expanded in its current location, the system can relocate it to another suitable area.

Example:

Old Allocated Size: 30 KB

New Requested Size: 50 KB

Output:

Operation : RELOCATED

Allocation Type : NON-CONTIGUOUS

Segments Used : 2

This demonstrates how an allocation may need to be moved when sufficient contiguous space is unavailable.

5.7 Internal Fragmentation

Internal fragmentation occurs when the amount of memory allocated to a request is larger than the amount actually requested.

The project demonstrates this using an allocation unit of 10 KB.

For example:

Requested Size = 51 KB

Allocated Size = 60 KB

Therefore:

Internal Waste = 60 - 51

= 9 KB

The internal fragmentation percentage is calculated as:

Internal Fragmentation =

(Internal Waste / Allocated Memory) × 100

For example:

Allocated Memory = 80 KB

Internal Waste = 9 KB

Internal Fragmentation =

(9 / 80) × 100

= 11.25%

This provides a clear demonstration of how unused space can exist inside an allocated block.

5.8 External Fragmentation

External fragmentation occurs when free memory exists but is distributed across multiple separate free blocks.

The system reports:

Number of Free Blocks

Total Free Memory

Largest Free Block

External Fragmentation

For example, if:

Total Free Memory = 40 KB

Largest Free Block = 20 KB

then although 40 KB is available in total, a single 40 KB contiguous request cannot necessarily be satisfied.

This demonstrates the difference between total available memory and the largest immediately usable free block.

5.9 Fragmentation Analysis

The system provides a dedicated:

4. Fragmentation Analysis

option.

It reports both:

External Fragmentation

Number of Free Blocks

Total Free Memory

Largest Free Block

External Fragmentation %

Internal Fragmentation

Allocated Memory

Unused Space Inside Allocated Blocks

Internal Fragmentation %

This gives the user a combined view of memory efficiency.

5.10 Memory Utilization

Memory utilization indicates how much of the total memory is currently being used.

The general calculation is:

Utilization =

(Used Memory / Total Memory) × 100

For example:

Used Memory = 80 KB

Total Memory = 100 KB

Utilization = 80%

5.11 Allocation Unit

To demonstrate internal fragmentation clearly, the implementation uses a 10 KB allocation unit.

For a request that is not an exact multiple of 10, the allocated size is rounded upward.

Examples:

20 KB → 20 KB

35 KB → 40 KB

51 KB → 60 KB

Therefore:

35 KB request → 5 KB internal waste

51 KB request → 9 KB internal waste

This is a controlled simulation mechanism used to make internal fragmentation visible during the project demonstration.

---

6. Test Cases and Results

The following test cases were used to validate the implemented functionality.

Test Case 1 — Basic Allocation

Input:

Total Memory = 100 KB

Strategy = First Fit

Request = 20 KB

Expected Result:

A 20 KB block should be allocated.

Observed Result:

Allocation ID : A001

Requested Size : 20 KB

Allocated Size : 20 KB

Internal Waste : 0 KB

Allocation Type: CONTIGUOUS

Status: PASS

Test Case 2 — Internal Fragmentation

Input:

Request = 51 KB

Because the allocation unit is 10 KB:

Requested Size = 51 KB

Allocated Size = 60 KB

Therefore:

Internal Waste = 9 KB

Observed Result:

Internal Waste : 9 KB

Status: PASS

Test Case 3 — Deallocation

Several blocks were allocated and one allocation was released.

Example:

A001 → 20 KB

A002 → 20 KB

A003 → 20 KB

Then:

Free A002

Observed Result:

Allocation A002 freed successfully.

The corresponding block becomes FREE.

Status: PASS

Test Case 4 — Memory Status

After allocation and deallocation, the system was tested using:

3. Display Memory Status

The system correctly displayed:

Offset

Block Size

USED/FREE status

Allocation ID

Allocation Type

Used Memory

Free Memory

Total Memory

Utilization

Status: PASS

Test Case 5 — Fragmentation Analysis

The fragmentation-analysis option was tested after allocation and deallocation.

The system displayed:

EXTERNAL FRAGMENTATION

Number of Free Blocks

Total Free Memory

Largest Free Block

External Fragmentation

and:

INTERNAL FRAGMENTATION

Allocated Memory

Unused Space Inside Allocated Blocks

Internal Fragmentation

Status: PASS

Test Case 6 — Reallocation / Shrinking

An allocation of 40 KB was resized to 20 KB.

Observed Result:

Operation : SHRINK

Old Allocated Size: 40 KB

New Requested Size: 20 KB

New Allocated Size: 20 KB

The released memory was returned to the available memory pool.

Status: PASS

Test Case 7 — Reallocation / Expansion

An allocation of 20 KB was expanded to 40 KB when sufficient adjacent memory was available.

Observed Result:

Operation : EXPAND IN PLACE

Old Allocated Size: 20 KB

New Requested Size: 40 KB

New Allocated Size: 40 KB

Status: PASS

Test Case 8 — Reallocation / Relocation

A block was requested to increase from 30 KB to 50 KB when sufficient contiguous space was not available at its current location.

Observed Result:

Operation : RELOCATED

Old Allocated Size: 30 KB

New Requested Size: 50 KB

New Allocated Size: 50 KB

Allocation Type : NON-CONTIGUOUS

Segments Used : 2

This demonstrates the system's ability to handle a larger request when the original location cannot accommodate it.

Status: PASS

Test Case 9 — Non-Contiguous Allocation

The system was tested with a large request after creating separated free regions.

Observed Result:

Allocation Type: NON-CONTIGUOUS

Segments Used : 2

This demonstrates allocation across multiple memory segments.

Status: PASS

Test Case 10 — Invalid Menu Input

An invalid menu option was entered:

Enter your choice: 20

The system responded:

Invalid choice. Please try again.

Status: PASS

---

7. Conclusion

The **Linux Dynamic Memory Allocation and Memory Monitoring System** successfully demonstrates important memory-management concepts through a practical C-based Linux application.

The implemented system provides a controlled memory pool in which allocation, deallocation, and reallocation operations can be performed and monitored. First Fit, Best Fit, and Worst Fit strategies provide different approaches for selecting suitable free memory blocks.

The project also demonstrates both contiguous and non-contiguous allocation and provides allocation IDs for tracking individual memory requests.

A major feature of the system is its fragmentation analysis. External fragmentation is analyzed by examining the distribution of free memory blocks, while internal fragmentation is demonstrated using a 10 KB allocation unit. This makes the otherwise difficult-to-observe concept of unused space inside allocated blocks clearly visible.

The reallocation module further demonstrates three practical situations: shrinking an allocation, expanding it in place, and relocating it when sufficient space is not available at the original location.

Overall, the project provides practical understanding of:

- Dynamic memory allocation
- Memory block management
- First Fit, Best Fit, and Worst Fit
- Contiguous and non-contiguous allocation
- Memory deallocation
- Memory reallocation
- Internal fragmentation
- External fragmentation
- Memory utilization
- Linux-based C programming
- GCC compilation and execution

The project therefore serves as a practical demonstration of Operating Systems memory-management concepts and provides a foundation for further extensions such as Linux `/proc`-based monitoring, detailed leak detection, workload-based performance comparison, and report generation.